

Judge Coverage

Why This Matters

The external Judge is intended to test translated code with evidence that was not used during repair. For that result to be meaningful, the documentation must distinguish a real system-test suite from a public example in a problem statement.

The Judge execution code and the local AOJ system-suite corpus are implemented. The remaining questions are which problems qualify after local C-baseline validation and how to treat problems without reproducible system coverage.

What Comes with Project CodeNet

CodeNet contains source submissions, acceptance status, runtime and memory metadata, problem descriptions, and usually an input/output example. The example in `derived/input_output` is the first sample extracted from the problem description.

CodeNet does not include the complete suites that AIZU and AtCoder originally used to assign AC, WA, TLE, and other verdicts. An experiment that uses only the CodeNet sample can therefore claim only **sample-I/O evaluation**, not broad system-test or hidden-test coverage.

This distinction corrects an earlier assumption in the project that the CodeNet-derived files represented a comprehensive Judge corpus.

What the Current Evaluator Already Does

Given a valid suite, the Evaluator:

1. compiles and runs the original C program;
2. confirms that C is accepted on the local cases;
3. runs Rust on exactly the same cases;
4. applies the problem's time and memory limits;
5. records per-case verdict counts and the first failure;
6. keeps initial and final Rust observations separate.

Rust is not treated as failed when the C baseline, test layout, or execution environment is invalid. These situations are reported separately.

Available Path for AIZU Problems

AOJ exposes problem descriptions and system-test inputs and outputs through public endpoints. CodeNet problem IDs are anonymized; their numeric suffix is not the original AOJ problem ID.

The accepted-C subset can instead be mapped through exact problem-description content.

`prepare_aoj_problem_mapping.py` downloads and caches the AOJ catalog and available English and Japanese descriptions, calculates SHA-256 hashes, and accepts only one-to-one exact matches. Titles, limits, numeric suffixes, source comments, and program behavior are not mapping criteria.

The accepted-C population contains 1,874 AIZU problems. Of these, 1,832 have a CodeNet problem description that can participate in exact-hash mapping; 42 have no description and remain unmapped unless an authoritative external relation becomes available. The completed mapping found 1,763 exact one-to-one matches. The other 69 described problems had no exact AOJ description match.

For implementation purposes, AOJ offers:

```
/testcases/{problem_id}/header  
/testcases/{problem_id}/{case_serial}
```

The case response normally includes input and output together. The extractor must compare retrieved byte sizes with the header because large combined responses can be truncated. Separate input and output requests are the fallback when validation fails.

Current Retrieved Coverage

An earlier prefetch reported 1,487 complete directories and 29,343 case pairs, but it derived AOJ IDs from CodeNet numeric suffixes. Those files are internally integrity-validated but associated with the wrong CodeNet problems in many cases. They are not valid Judge coverage and must not be used as research evidence under their existing directories.

The exact mapping and mapped-suite prefetch are complete. They produced 1,763 mapped problems; 1,738 have downloaded system suites totaling 33,301 cases, while four downloads failed and 21 problems were unavailable. These files are the structural candidate population, not the final Judge-backed population. A later accepted-C baseline pass will define the final Judge-backed population.

For Judge-enabled runs, OxCidation applies structural eligibility before sampling: the problem must have metadata limits and a complete, non-empty set of matched `.in` / `.out` files. This avoids selecting known uncovered problems, but does not replace accepted-C baseline execution.

At least one accepted C program must compile locally and pass the downloaded suite before a problem is classified as Judge-backed. Local visible-test preparation uses a fixed `gnu11` then `gnu89` compatibility sequence and records the successful profile. A source that fails both is explicitly reported as `baseline_compile_failed` and supplies no visible-test evidence. Special judges, multiple valid outputs, historical compiler behavior, or changed problem data may reduce the final eligible population. The current Evaluator performs this check during an experiment, but a population-wide C-baseline pass before sampling would make the final sampling frame explicit.

Extraction Implementation

One suite is stored per problem and reused for every selected C program and prompt for that problem.

The implemented prefetcher uses bounded problem-level concurrency with one global request-rate limit. It is resumable and validates file sizes and SHA-256 hashes before publishing a suite. Unavailable cases must use a deterministic reserve policy defined before any model result is observed.

`prepare_aoj_system_tests.py` requires `mapped_problems.csv` and reads the explicit `aoj_problem_id` from each row. It cannot infer an AOJ ID from a CodeNet ID. The command, output layout, restart behavior, and targeted retry mode are documented in the project README.

Earlier alternative: lazy retrieval

Lazy retrieval was considered as a way to fetch a suite only when an experiment first reached its problem, then cache it for all programs and prompts. It would have spread API traffic and avoided requests for problems never reached, but it would also have made network availability part of experiment execution.

The current plan uses a resumable prefetch after exact mapping. Lazy retrieval remains an alternative for a future corpus or currently unavailable cases. Any such use would still require a frozen problem manifest, rate

limiting, atomic writes, hashes, and explicit retrieval-failure reporting.

AtCoder Limitation

AtCoder historically published system tests, but publication was suspended and equivalent current access cannot be assumed. CodeNet does not solve this gap.

Until a permitted source is established, AtCoder problems must be reported as sample-only or without external Judge coverage. Their results must not be mixed with AOJ system-suite results as if both used equivalent evidence.

Cases That Need Special Treatment

Downloading input and expected output is not always enough to reproduce an online judge. The following may require explicit support or exclusion:

- interactive problems;
- special judges with multiple valid outputs;
- floating-point tolerance;
- output-only tasks;
- historical changes to problem data or compiler behavior.

An accepted C reference should be run against every retrieved suite before the problem is classified as Judge-backed.

Coverage That Should Appear in Reports

Each result should state:

Field	Example values	Purpose
Judge source	aoj_system, statement_sample, unavailable	Identifies where the cases came from.
Coverage class	system_suite, public_sample, none	Prevents unlike evidence from being combined.
Case count	Integer	Shows how much executable evidence was used.

External References

- [Project CodeNet repository and dataset description](#)
- [AOJ API reference](#)
- [online-judge-tools system-case support](#)
- [AtCoder suspension of test-case publication](#)
- [AtCoder current information page](#)